

El clasificador de solicitudes

PF1821 · Construcción de Agentes y Automatización con Herramientas Low Code
Módulo 2 · MA04560 · Fundamentos de n8n y manipulación de datos

MODALIDAD

Individual, asincrónica.

APRENDIZAJE ESPERADO 4

Implementar lógica condicional y estructuras de control en workflows, aplicando buenas prácticas de debugging y trazabilidad.

CRITERIOS DE EVALUACIÓN

- 4.1 Configura workflows con múltiples rutas condicionales.
- 4.2 Utiliza operadores lógicos y comparaciones en expresiones n8n.
- 4.3 Aplica herramientas de debugging para identificar errores básicos en workflows.

Contexto

El servicio técnico Tecnofix (empresa ficticia) repara computadores y recibe solicitudes por un formulario. Hoy una persona lee cada solicitud y la reenvía al área que corresponde. Tecnofix quiere que un workflow de n8n las clasifique solo.

Reglas de clasificación

Ruta	Condición
Urgente	<code>tipo</code> es "falla" y (el <code>plan</code> es "empresa" o <code>dias_sin_servicio</code> es 2 o más)
Garantía	<code>tipo</code> es "falla" y <code>en_garantia</code> es verdadero, y no es urgente
Ventas	<code>tipo</code> es "cotización"
Revisión manual	Cualquier otro caso

Insumos

Estas son las seis solicitudes de prueba. Fíjalas como datos del trigger con *Pin data* para probar sin volver a enviar el formulario:

```
[
  { "id": 1, "tipo": "falla",      "plan": "empresa", "dias_sin_servicio": 0,
    "en_garantia": false },
  { "id": 2, "tipo": "falla",      "plan": "hogar",    "dias_sin_servicio": 1,
    "en_garantia": true  },
  { "id": 3, "tipo": "FALLA",      "plan": "hogar",    "dias_sin_servicio": "3",
    "en_garantia": false },
  { "id": 4, "tipo": "cotización", "plan": "hogar",    "dias_sin_servicio": 0,
    "en_garantia": false },
  { "id": 5, "tipo": "",           "plan": "empresa", "dias_sin_servicio": 5,
    "en_garantia": false },
  { "id": 6, "tipo": "falla",      "plan": "hogar",    "dias_sin_servicio": 0,
    "en_garantia": false }
]
```

Qué tienes que hacer

- 1. Predice.** Antes de construir, escribe en una tabla a qué ruta debería ir cada solicitud según las reglas.
- 2. Normaliza.** Con *Edit Fields (Set)*, deja `tipo` en minúsculas y sin espacios y convierte `dias_sin_servicio` a número.
- 3. Enruta (criterios 4.1 y 4.2).** Configura un *Switch* con una salida por ruta. Ordena las reglas: el Switch envía cada ítem a la primera que se cumple. La regla de "Urgente" combina **Y** y **O**; como cada regla del Switch usa un solo combinador, escríbela como una expresión con `&&` y `||` que dé verdadero o falso (condición de tipo *Boolean*, *is true*). Activa la salida de respaldo (*Fallback Output*) para "Revisión manual".
- 4. Marca la ruta.** Al final de cada salida, agrega un *Edit Fields (Set)* que escriba el campo `ruta` con el nombre de la ruta.
- 5. Depura (criterio 4.3).** Ejecuta con las seis solicitudes y completa tu tabla con la ruta obtenida. Si alguna no coincide con tu predicción, usa la vista de entrada y salida de cada nodo y el historial de *Executions* para encontrar la causa. Anota el síntoma, la causa y la corrección.
- 6. Casos límite.** Explica en 3 a 5 líneas qué pasaría con las solicitudes 3 y 5 si no hubieras normalizado los datos ni activado la salida de respaldo.

Entrega

Sube a esta Tarea el workflow exportado en JSON, tu tabla con la ruta predicha y la obtenida para cada solicitud, el registro de depuración del punto 5 y tu respuesta al punto 6.